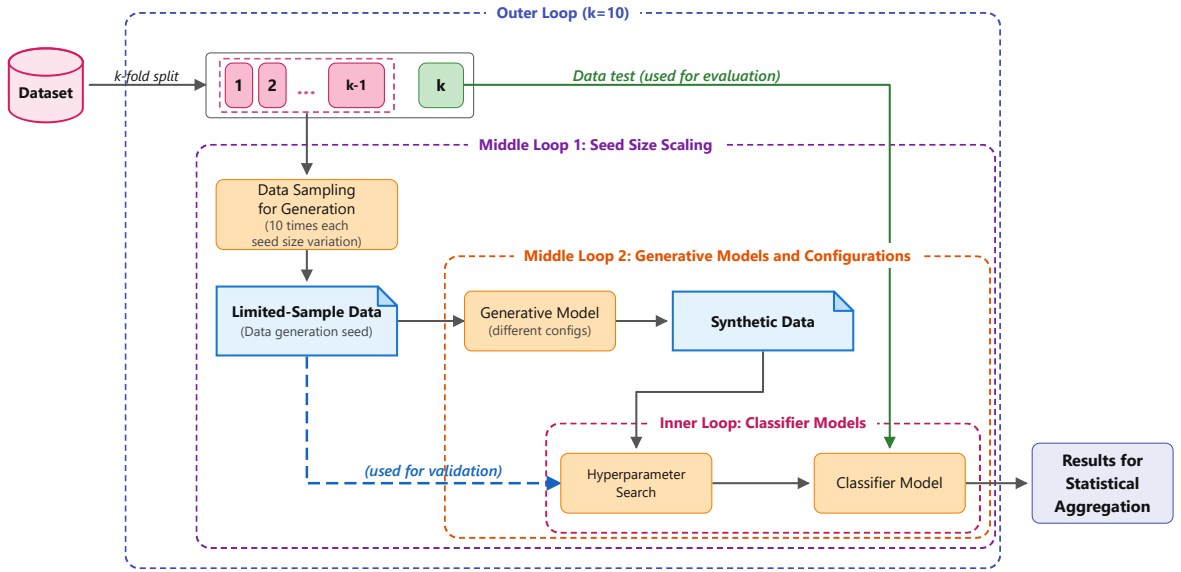
Synthetic Time-Series Data Generation Report (gen_params_10)

5 Datasets | 10 Folds × 10 Random Seed Selections | Seed Sizes: 1, 2, 5, 10, 20 | Scaling Factors: 2x, 3x, 5x, 10x | 5 Classifiers | 10 Generators

Experimental Pipeline & Architecture Flowchart

End-to-end multi-loop evaluation workflow across k-fold cross-validation, seed size variations, generative hyperparameter configurations, and classifier hyperparameter tuning.



Question 1 (Q1): Baseline Performance & Hyperparameter Tuning Gains

How do machine learning models perform across different initial seed sizes (1, 2, 5, 10, 20) before and after classifier hyperparameter tuning (Baseline vs. Tuned Baseline)?

- **Substantial Tuning Gains for RNN (Seed Size ≥ 2):** With default parameters, RNN achieves an overall mean accuracy of **49.17%**. Classifier hyperparameter tuning lifts RNN accuracy to **51.86% (+9.71%)** at Seed = 2, **58.63% (+14.41%)** at Seed = 5, and **69.53%** at Seed = 20.
- **One-Shot Regime (Seed Size = 1):** KNN achieves the highest 1-shot accuracy at **70.65%** (using 1-nearest neighbor), while RNN remains at **41.23%**.
- **Strong Traditional Baselines:** KNN (mean **82.89%**) and RandomForest (mean **80.86%**) provide the highest baseline accuracy on unaugmented data, exceeding **93%** at Seed = 20.

Table 1.1: Mean Accuracy of Baseline and Tuned Baseline by Seed Size

	SeedsPerLabel	1	2	5	10	20	Overall_Mean
Model	Method						
KNN	Baseline	0.7032	0.7625	0.8443	0.8950	0.9355	0.8281
	Tuned_Baseline	0.7065	0.7611	0.8455	0.8950	0.9363	0.8289
LinearSVC	Baseline	0.6599	0.7055	0.7642	0.7874	0.7963	0.7427
	Tuned_Baseline	0.6313	0.6914	0.7558	0.7839	0.7934	0.7312
LogisticRegression	Baseline	0.6841	0.7327	0.7967	0.8243	0.8388	0.7753
	Tuned_Baseline	0.6870	0.7245	0.8023	0.8305	0.8448	0.7778
RNN	Baseline	0.4123	0.4215	0.4422	0.5441	0.6386	0.4917
	Tuned_Baseline	0.4123	0.5186	0.5863	0.6382	0.6953	0.5701
RandomForest	Baseline	0.6663	0.7326	0.8309	0.8878	0.9308	0.8097
	Tuned_Baseline	0.6667	0.7301	0.8297	0.8868	0.9299	0.8086

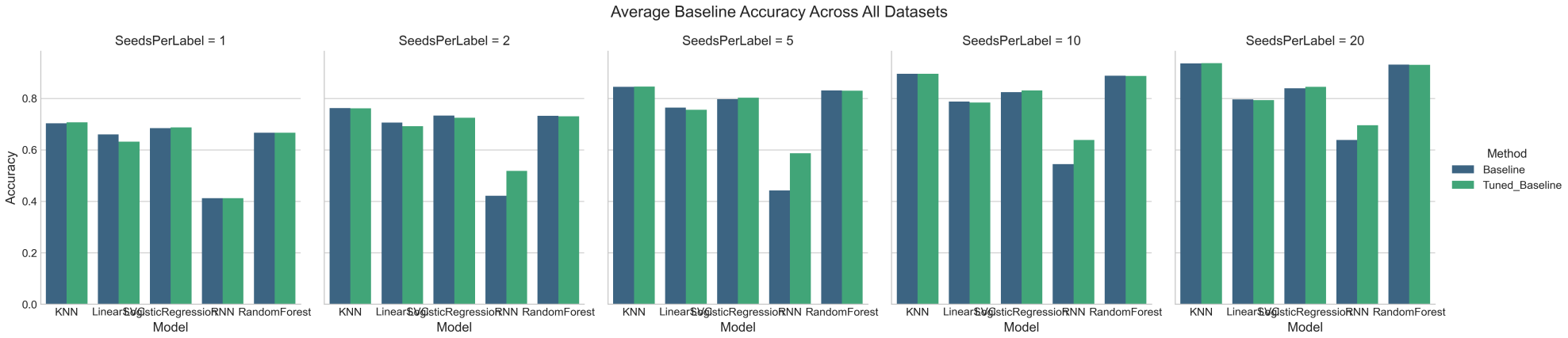


Figure 1.0: Average Baseline vs. Tuned Baseline Accuracy Across All 5 Datasets

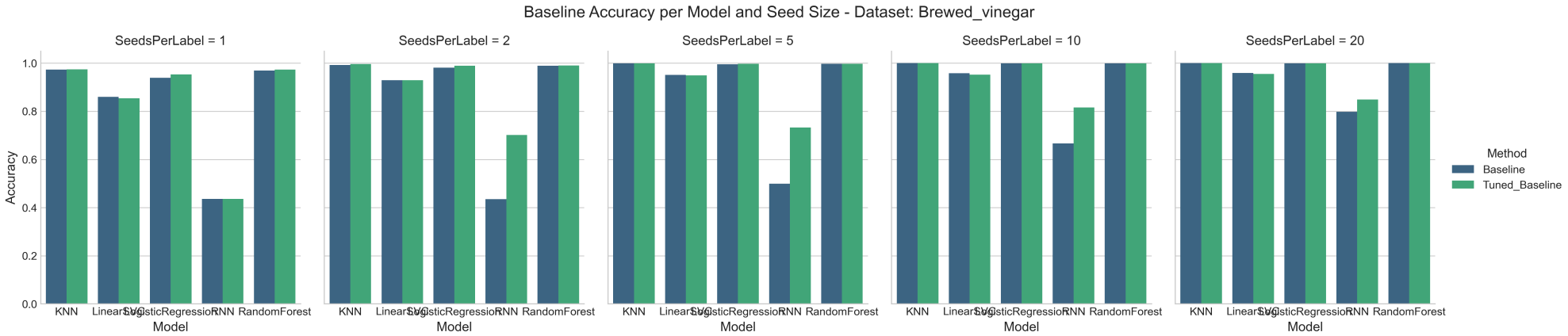


Figure 1.1: Baseline vs. Tuned Baseline Accuracy — Dataset: Brewed_vinegar

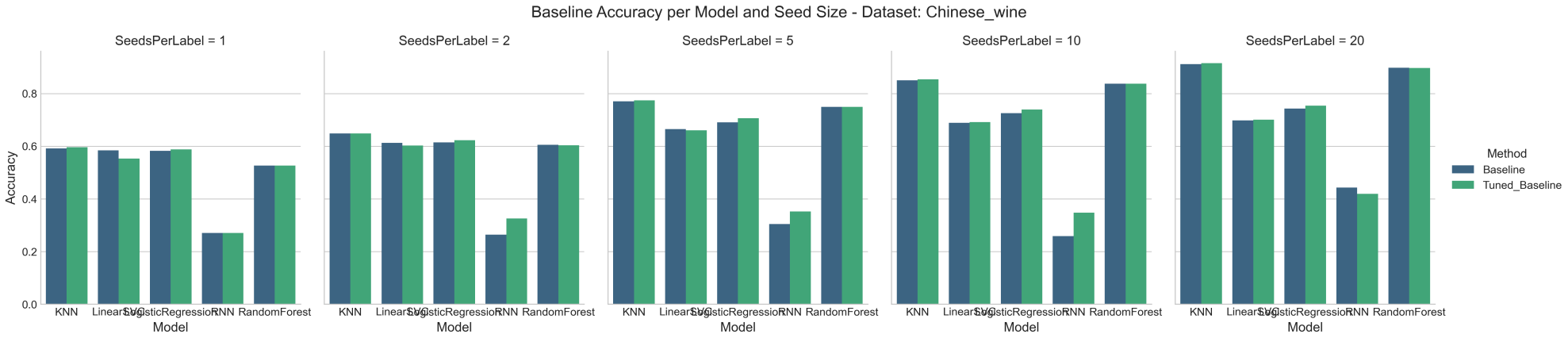


Figure 1.2: Baseline vs. Tuned Baseline Accuracy — Dataset: Chinese_wine

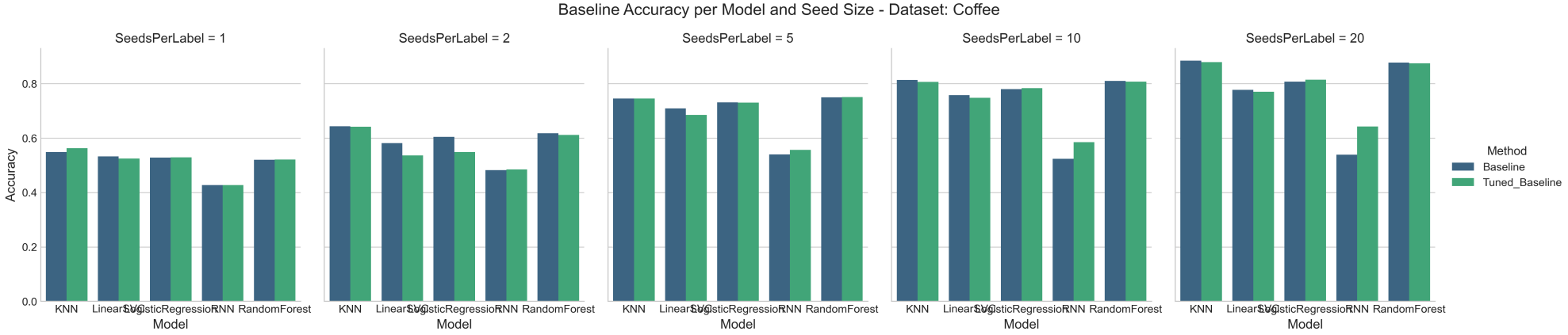


Figure 1.3: Baseline vs. Tuned Baseline Accuracy — Dataset: Coffee

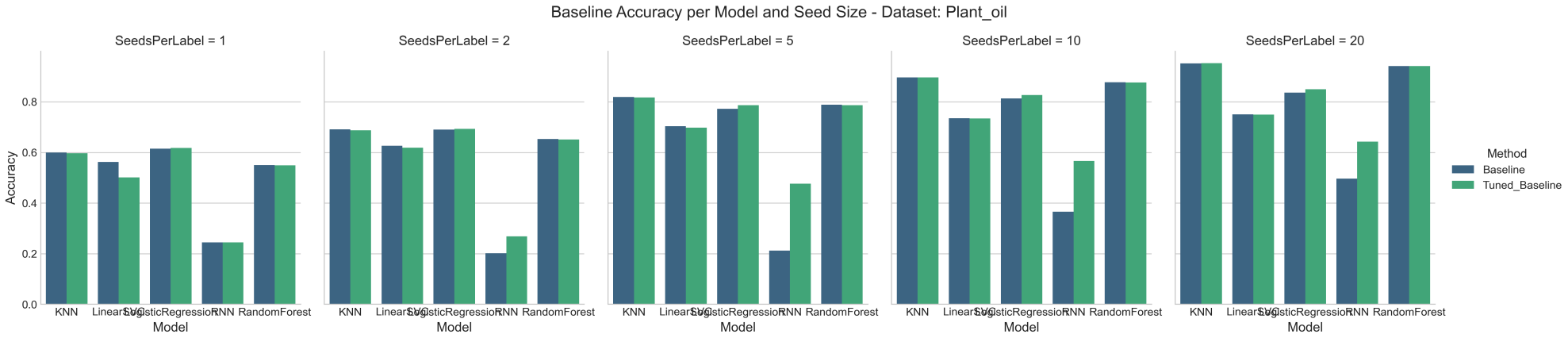


Figure 1.4: Baseline vs. Tuned Baseline Accuracy — Dataset: Plant_oil

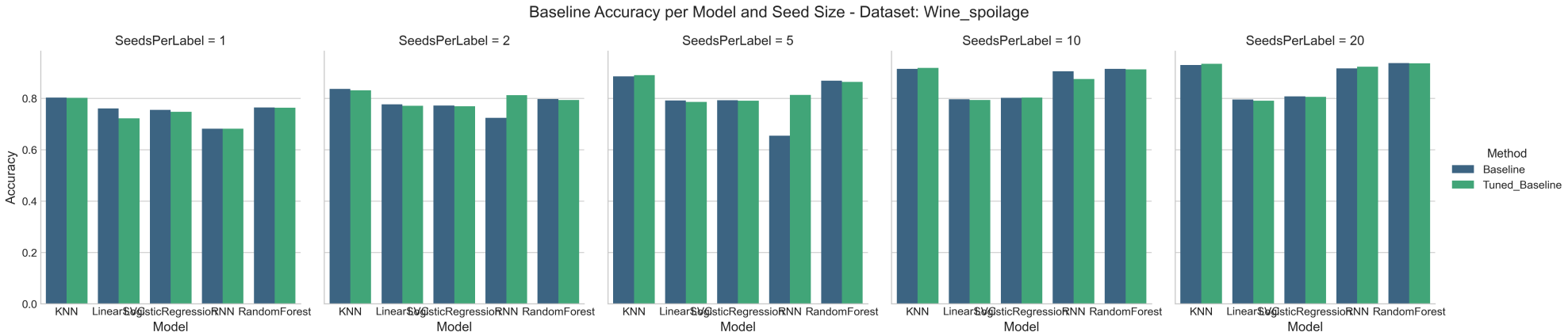


Figure 1.5: Baseline vs. Tuned Baseline Accuracy — Dataset: Wine_spoilage

Question 2 (Q2): Effect of Synthetic Data on Machine Learning Models

How does each synthetic data generation method impact different ML classifiers relative to the Default Baseline (Q2.1) and the Tuned Baseline (Q2.2)?

- **Major Improvements on RNN:** Synthetic data augmentation improves RNN accuracy by **+12.77% to +20.31%** over Default Baseline (all generators except TimeVAE, which gains +5.97%), and provides a net gain of **+10.48% (Mixup)**, **+10.45% (GMM)**, **+10.10% (SMOTE)**, and **+8.99% (Time_Warping)** above Tuned Baseline.
- **Consistent Gains Across All Five Classifiers:** Six methods (Jittering, Scaling, Magnitude_Warping, SMOTE, Mixup, GMM) deliver positive mean net gains (ΔAcc > 0) on all five classifiers compared to Tuned Baseline. Jittering leads on RandomForest (**+1.98%**), LinearSVC (**+1.68%**) and LogisticRegression (**+0.73%**), GMM leads on KNN (**+0.99%**), and Mixup leads on RNN (**+10.48%**). For the traditional classifiers these gains are small (mostly below 2%).
- **Few-Shot Limitations of TimeVAE:** Training a variational autoencoder on very small seed sets (2–20 samples per class) causes overfitting, slightly helping RNN (+0.30% vs. Tuned Baseline) but reducing accuracy on LinearSVC (-6.70%) and LogisticRegression (-5.29%).

Note on generator-configuration selection: Q2–Q4, Q6 and Q7 use the best generator configuration per (Dataset, Model, Seed Size, Scale Factor), selected by mean F1 on the same test folds that are used for evaluation. These figures are therefore an optimistic upper bound. If results are instead averaged over all tested configurations, the net gain over Tuned Baseline on RNN stays clearly positive for 8 of 10 generators (**+6.2% to +9.4%**; HMM_GMM −2.2%, TimeVAE −7.0%), whereas on KNN, LinearSVC, LogisticRegression and RandomForest most gains shrink to within ±1.5% and often turn negative. Mixup is the only generator that stays positive on all five classifiers under this averaging.

Table 2.1: Mean Accuracy Gain vs. Default Baseline (ΔAcc)

Method	Jittering	Scaling	Magnitude_Warping	Time_Warping	SMOTE	ADASYN	Mixup	GMM	HMM_GMM	TimeVAE
Model										
KNN	0.0045	0.0022	0.0030	0.0003	0.0097	0.0084	0.0103	0.0106	0.0013	-0.0070
LinearSVC	0.0053	-0.0034	-0.0001	-0.0183	-0.0006	0.0048	0.0015	0.0040	-0.0032	-0.0699
LogisticRegression	0.0098	0.0050	0.0059	-0.0071	0.0071	0.0055	0.0092	0.0091	0.0045	-0.0469
RNN	0.1561	0.1561	0.1616	0.1683	0.1993	0.1426	0.2031	0.2029	0.1277	0.0597
RandomForest	0.0187	0.0059	0.0105	-0.0099	0.0039	0.0053	0.0051	0.0096	0.0036	-0.0091

Table 2.2: Net Accuracy Gain vs. Tuned Baseline (ΔAcc)

Method	Jittering	Scaling	Magnitude_Warping	Time_Warping	SMOTE	ADASYN	Mixup	GMM	HMM_GMM	TimeVAE
Model										
KNN	0.0037	0.0014	0.0022	-0.0005	0.0091	0.0097	0.0096	0.0099	0.0006	-0.0078
LinearSVC	0.0168	0.0081	0.0114	-0.0067	0.0044	0.0079	0.0065	0.0090	0.0017	-0.0670
LogisticRegression	0.0073	0.0025	0.0034	-0.0096	0.0012	-0.0031	0.0033	0.0032	-0.0014	-0.0529
RNN	0.0777	0.0777	0.0832	0.0899	0.1010	0.0833	0.1048	0.1045	0.0294	0.0030

Method	Jittering	Scaling	Magnitude_Warping	Time_Warping	SMOTE	ADASYN	Mixup	GMM	HMM_GMM	TimeVAE
Model										
RandomForest	0.0198	0.0069	0.0116	-0.0089	0.0049	0.0070	0.0061	0.0106	0.0047	-0.0082

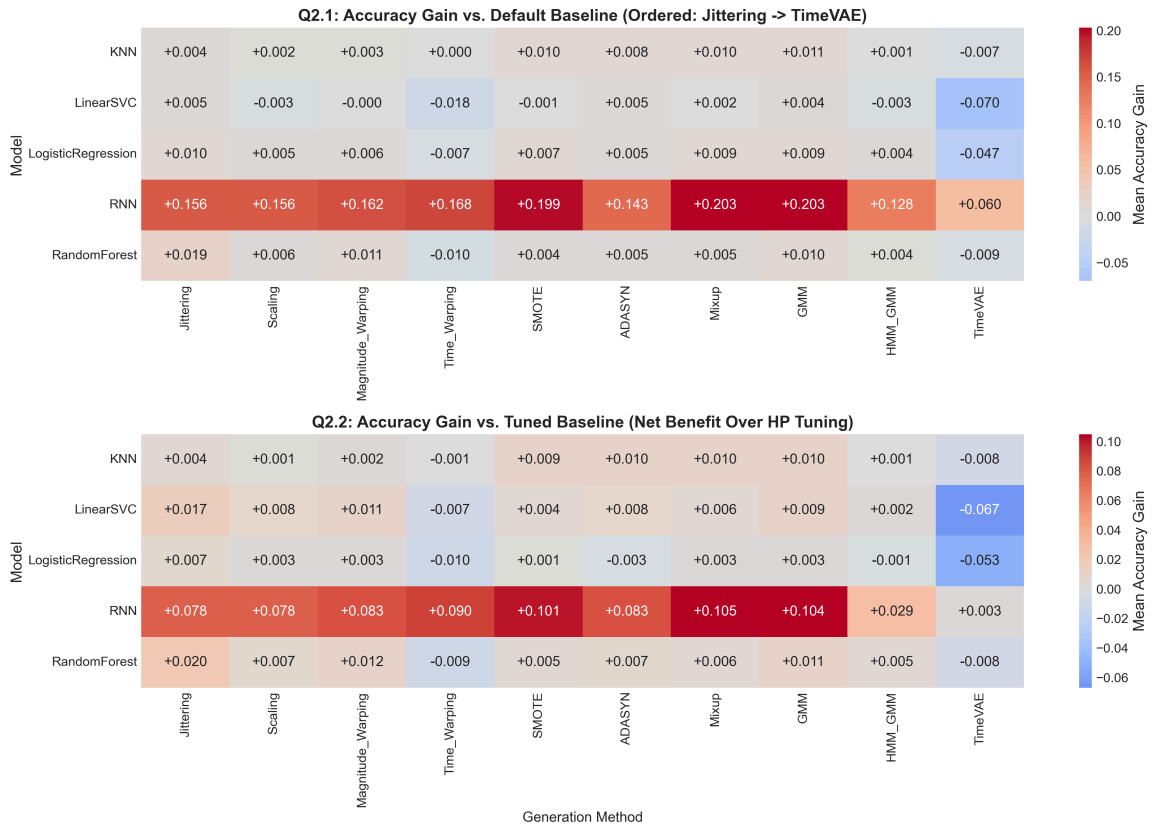


Figure 2.1: Accuracy Gain Matrix (ΔAcc) vs. Default Baseline (top) and vs. Tuned Baseline (bottom)

Question 3 (Q3): Scaling Effect by Seed Size (1x to 10x)

How does expanding synthetic data volume from 1x (seed baseline) up to 2x, 3x, 5x, and 10x affect accuracy across seed sizes?

- **Continuous Scaling Growth on RNN:** For Seed ≥ 2, RNN accuracy increases steadily from 1x through 10x and exceeds the Tuned Baseline from 2x onward for every generator. At Seed = 1, accuracy dips slightly at 2x–3x and only rises clearly at 5x–10x (to about 50–51%).
- **Early Saturation (2x–3x) on Traditional Classifiers:** For KNN, LinearSVC, LogisticRegression, and RandomForest, most accuracy gains are achieved at 2x–3x scaling and remain stable through 10x.

Note on the ADASYN Curve at Seed Size = 10 and 20: ADASYN only generates synthetic samples when classes overlap near decision boundaries. Across the 5 datasets, ADASYN only produces samples on the 2 harder datasets with overlapping boundaries (*Chinese_wine* and *Coffee* at Seed = 10 and 20). Consequently, the ADASYN curve averages over those 2 harder datasets (where RNN 1x baseline is **39.11%** at Seed 10 and **49.08%** at Seed 20), whereas the other 9 methods average across all 5 datasets (**54.41%** at Seed 10 and **63.78%** at Seed 20). Within those 2 datasets, ADASYN strongly improves RNN accuracy from **39.11% (1x)** → **65.29% (10x)** at Seed 10 and **49.08% (1x)** → **65.82% (10x)** at Seed 20.

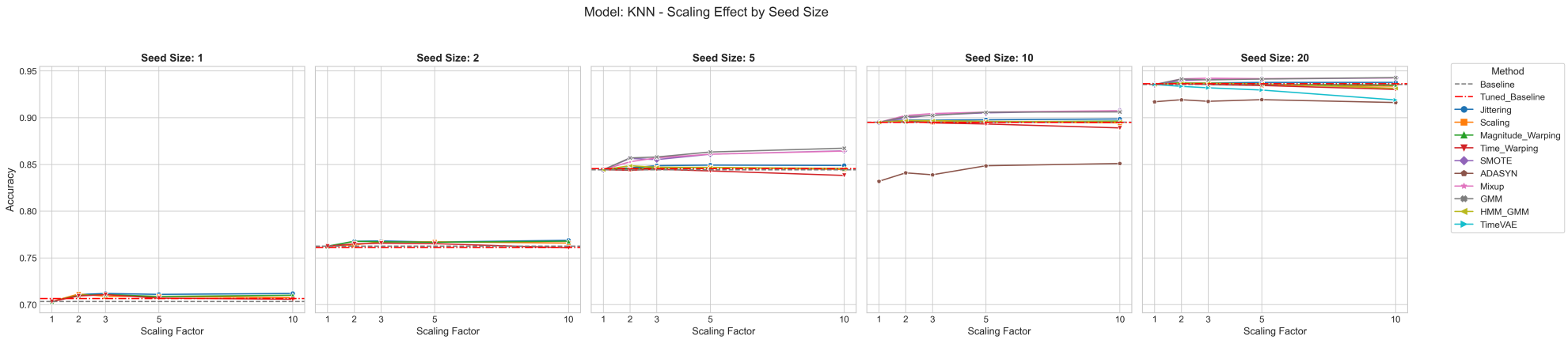


Figure 3.1: Scaling Effect (1x to 10x) Across Seed Sizes — Model: KNN

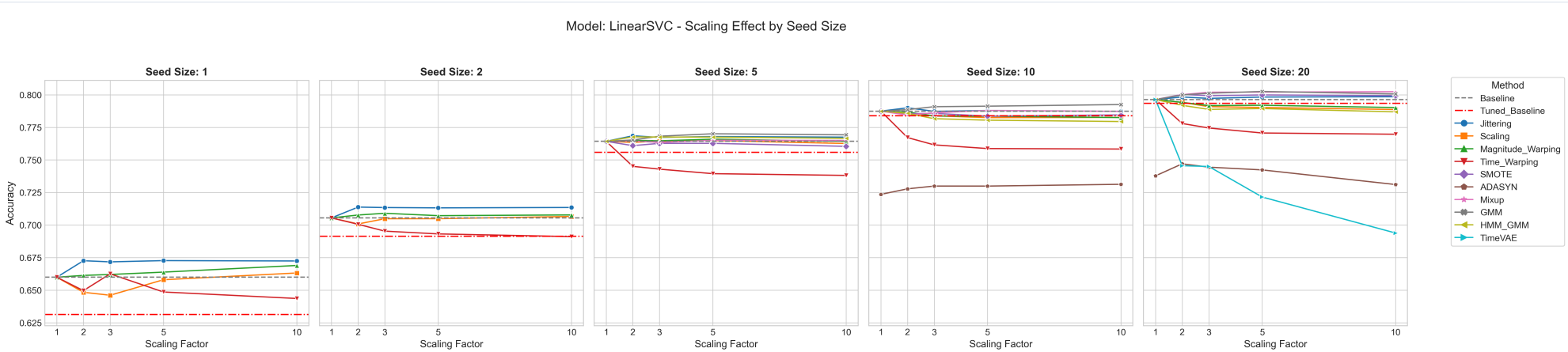


Figure 3.2: Scaling Effect (1x to 10x) Across Seed Sizes — Model: LinearSVC

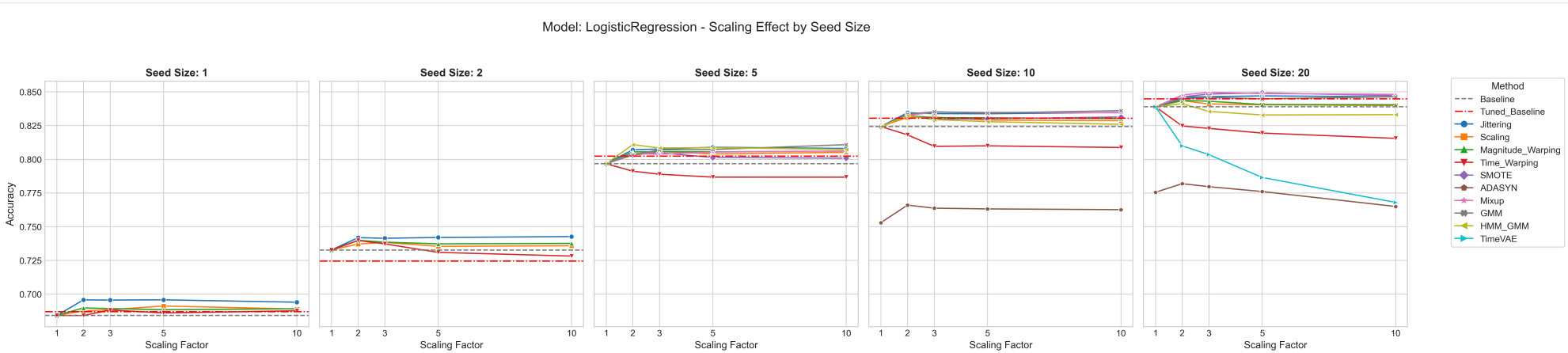


Figure 3.3: Scaling Effect (1x to 10x) Across Seed Sizes — Model: LogisticRegression

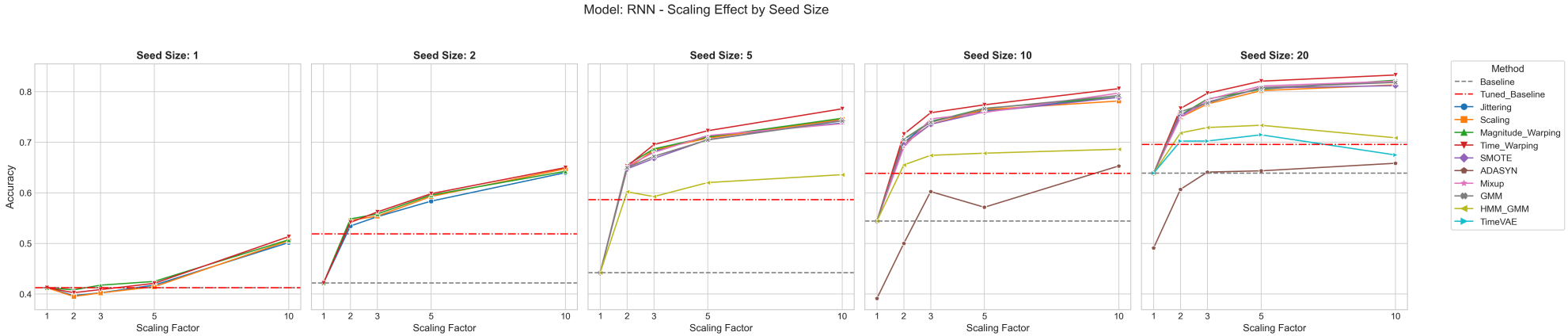


Figure 3.4: Scaling Effect (1x to 10x) Across Seed Sizes — Model: RNN

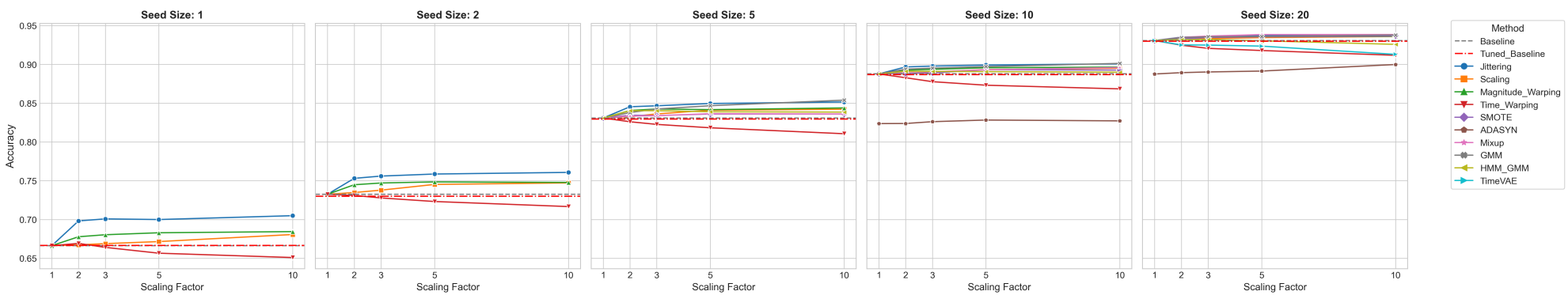


Figure 3.5: Scaling Effect (1x to 10x) Across Seed Sizes — Model: RandomForest

Question 4 (Q4): Effect of Seed Size per Generator Method

How does initial seed volume (1, 2, 5, 10, 20) influence method applicability and scaling trajectories from 1x to 10x?

- **Single-Sample Augmentations (Jittering, Scaling, Magnitude_Warping, Time_Warping):** Operate at all seed sizes including **Seed = 1**, lifting 1-shot RNN accuracy from 41.23% to **50.2%–51.3%** at 10x (best: Time_Warping, 51.32%).
- **Multi-Sample Generators:** SMOTE, Mixup, GMM and HMM_GMM produce results only from **Seed ≥ 5**; TimeVAE only at **Seed = 20**; ADASYN only at **Seed = 10 and 20**, and only on 2 of the 5 datasets (Chinese_wine, Coffee). Where available, SMOTE, Mixup and GMM show smooth upward trajectories.

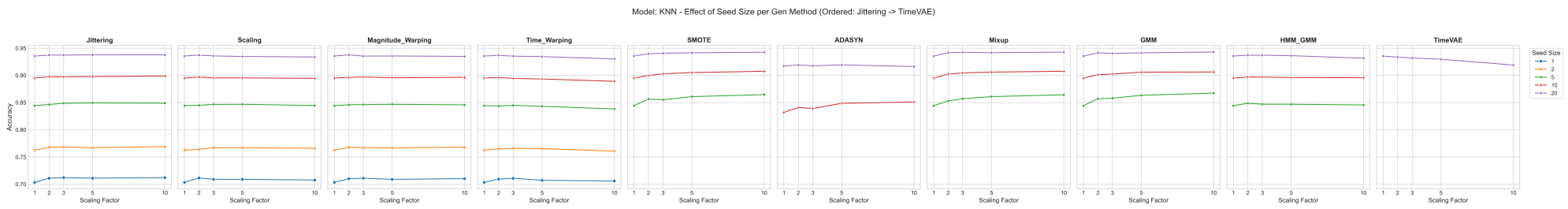


Figure 4.1: Effect of Seed Size (1 to 20) per Generator Method — Model: KNN



Figure 4.2: Effect of Seed Size (1 to 20) per Generator Method — Model: LinearSVC

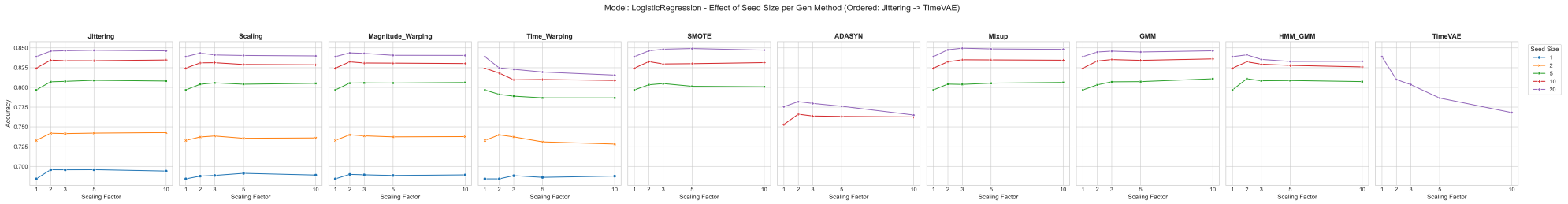


Figure 4.3: Effect of Seed Size (1 to 20) per Generator Method — Model: LogisticRegression

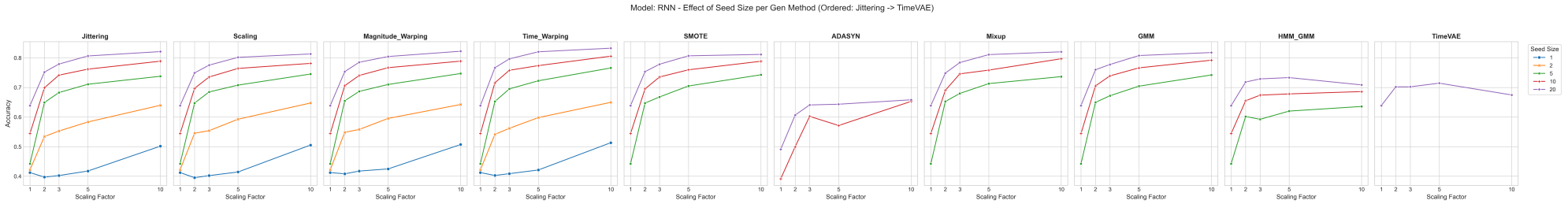


Figure 4.4: Effect of Seed Size (1 to 20) per Generator Method — Model: RNN

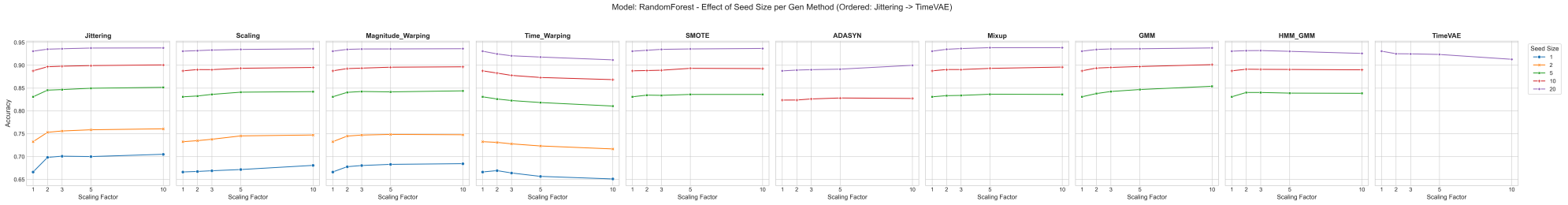


Figure 4.5: Effect of Seed Size (1 to 20) per Generator Method — Model: RandomForest

Question 5 (Q5): Generator Hyperparameter Sensitivity

How sensitive is classification accuracy to generator hyperparameter choices (5 configurations for 1-parameter methods, 10 configurations for multi-parameter methods)?

- **Most Stable Generators: Mixup** (spread of only **0.16%**, best at `alpha = 0.8` with **82.57%** accuracy), **SMOTE** (spread **0.29%**, best at `k_neighbors = 5` with **82.42%**), and **Jittering** (spread **0.71%**, best at `sigma = 0.05`) are highly robust to parameter selection.
- **Parameter-Sensitive Generators:** Signal-warping methods (Scaling, Magnitude_Warping, Time_Warping) require small perturbation magnitudes (`sigma = 0.05`); larger perturbations distort spectral peaks and reduce accuracy by **3.90%–4.44%**. For GMM, the best configuration is `n_components = 2, covariance_type = 'tied'` (**82.51%**); tied/full covariance configurations are nearly identical (82.37%–82.51%), while diag/spherical covariances perform worst (79.18%–81.82%).

Table 5.1: Optimal Generator Hyperparameters and Sensitivity Spread

Method	Optimal_Configuration	Mean_Acc	Std_Dev	Configs	Best_Acc	Worst_Acc	Spread
Jittering	{'sigma': 0.05}	0.7639	0.1768	5	0.7639	0.7568	0.0071
Scaling	{'sigma': 0.05}	0.7598	0.1773	5	0.7598	0.7208	0.0390
Magnitude_Warping	{'sigma': 0.05, 'knot': 4}	0.7607	0.1774	10	0.7607	0.7199	0.0408
Time_Warping	{'sigma': 0.05, 'knot': 8}	0.7507	0.1804	10	0.7507	0.7063	0.0444
SMOTE	{'k_neighbors': 5}	0.8242	0.1367	5	0.8242	0.8214	0.0029
ADASYN	{'n_neighbors': 5}	0.7640	0.1418	5	0.7640	0.7073	0.0567
Mixup	{'alpha': 0.8}	0.8257	0.1364	5	0.8257	0.8241	0.0016
GMM	{'n_components': 2, 'covariance_type': 'tied'}	0.8251	0.1364	10	0.8251	0.7917	0.0333
HMM_GMM	{'n_components': 1, 'n_mix': 1, 'covariance_type': 'tied'}	0.8042	0.1521	10	0.8042	0.7509	0.0532
TimeVAE	{'latent_dim': 8, 'epochs': 40, 'batch_size': 32}	0.7940	0.1582	10	0.7940	0.7639	0.0300

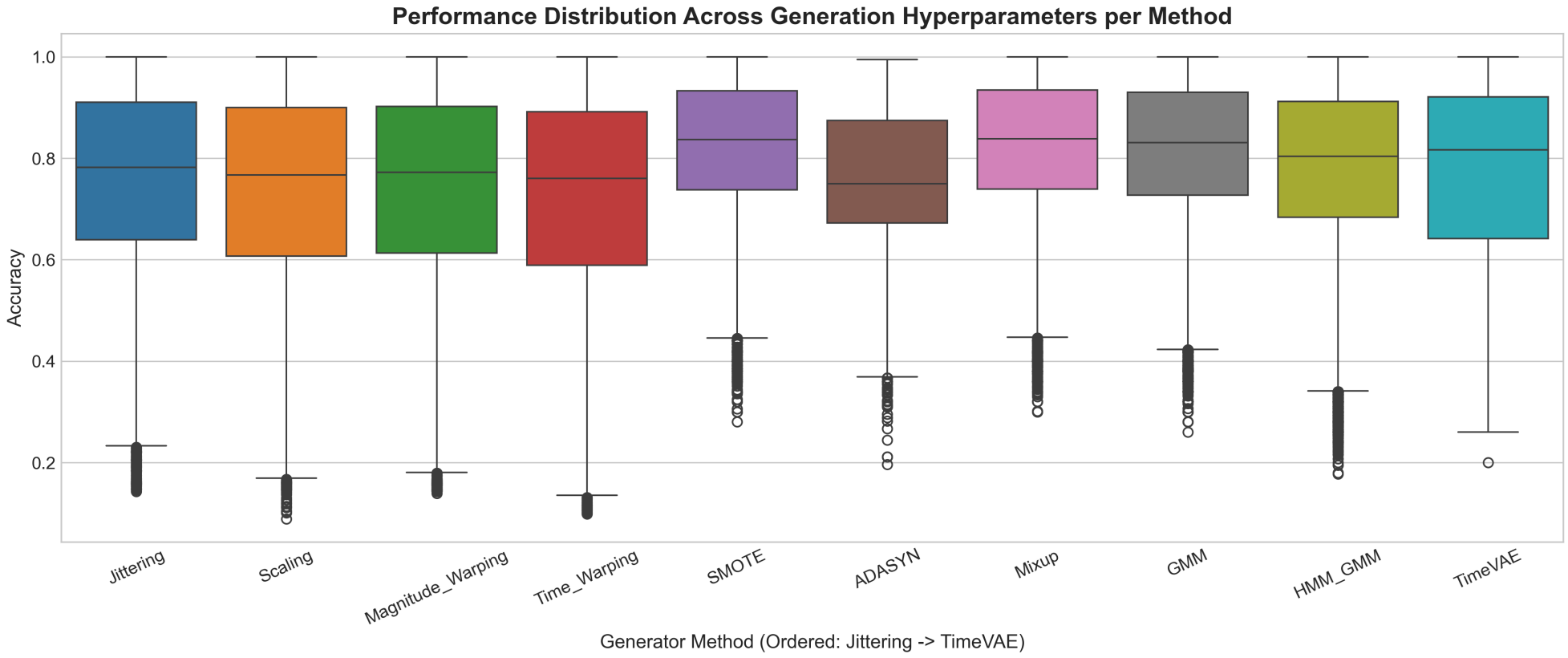


Figure 5.1: Accuracy Distribution Across Generator Hyperparameter Configurations

Question 6 (Q6): Statistical Significance Testing (Wilcoxon Signed-Rank Test)

Are accuracy gains over Tuned Baseline statistically significant ($p < 0.05$) across paired folds?

- **RNN:** 9 out of 10 generators (all except TimeVAE) achieve statistically significant gains (**$p < 0.0001$**) over Tuned Baseline, with win rates between **66.5% (HMM_GMM) and 91.8% (Mixup)**.
- **Traditional Classifiers:** Top generators (GMM, Mixup, SMOTE, Jittering) achieve statistically significant improvements (**$p < 0.05$**) on KNN, LinearSVC, and RandomForest (on RandomForest, Jittering reaches an **85.8%** win rate and GMM **78.0%**). Note that per-case gains for these classifiers are small (<2%), and p-values treat Scale Factors and Seed Sizes as separate paired observations, so they are not fully independent.

Table 6.1: Wilcoxon Signed-Rank Test p-values (vs. Tuned Baseline)

Method	Jittering	Scaling	Magnitude_Warping	Time_Warping	SMOTE	ADASYN	Mixup	GMM	HMM_GMM	TimeVAE
Model										
KNN	0.0	0.0005	0.0	0.0028	0.000	0.0000	0.0	0.0	0.9515	0.0000
LinearSVC	0.0	0.0000	0.0	0.1320	0.000	0.9742	0.0	0.0	0.7088	0.0000
LogisticRegression	0.0	0.9024	0.0	0.0000	0.706	0.0011	0.0	0.0	0.0028	0.0000
RNN	0.0	0.0000	0.0	0.0000	0.000	0.0000	0.0	0.0	0.0000	0.9348
RandomForest	0.0	0.0000	0.0	0.0000	0.000	0.0003	0.0	0.0	0.0000	0.0000

Table 6.2: Win Rate (%) Against Tuned Baseline

Method	Jittering	Scaling	Magnitude_Warping	Time_Warping	SMOTE	ADASYN	Mixup	GMM	HMM_GMM	TimeVAE
Model										
KNN	66.3	46.2	46.4	38.6	69.0	82.5	71.5	73.3	38.8	8.5
LinearSVC	84.5	60.8	68.6	50.8	62.8	49.4	69.5	71.8	48.7	11.5
LogisticRegression	69.7	48.8	56.2	44.7	46.2	33.1	55.2	61.7	45.2	6.5
RNN	77.3	77.9	84.5	81.6	91.7	79.1	91.8	91.7	66.5	48.0
RandomForest	85.8	73.5	83.5	23.1	63.8	69.1	65.8	78.0	65.8	19.0

Question 7 (Q7): Computational Cost vs. Accuracy Gain

What is the efficiency trade-off between execution time per fold (seconds) and net accuracy gain over Tuned Baseline?

- **Pareto-Optimal Generators: Jittering** (**1.87 s/fold**, +2.50% net gain) and **Magnitude_Warping** (**1.78 s/fold**, +2.24% net gain) are the fastest high-gain methods and support 1-shot learning. For Seed ≥ 5, **GMM** (**2.63 s/fold**, +2.75% net gain, **83.10%** mean accuracy) and **Mixup** (**2.49 s/fold**, +2.61% net gain, **82.96%** mean accuracy) deliver the highest overall accuracy.
- **Caveat:** Mean accuracy is not directly comparable across generators, because methods cover different seed sizes (e.g., GMM/Mixup/SMOTE start at Seed = 5, TimeVAE only at Seed = 20, while Jittering includes the hardest Seed = 1–2 cases). Net gain vs. Tuned Baseline is the fairer comparison.

Table 7.1: Execution Time and Accuracy Gain by Generator

Method	Mean_Time_Sec	Gain_vs_Baseline	Gain_vs_Tuned	Mean_Accuracy
Jittering	1.8726	0.0389	0.0250	0.7684
Scaling	1.7329	0.0332	0.0193	0.7627
Magnitude_Warping	1.7831	0.0362	0.0224	0.7657
Time_Warping	1.6892	0.0267	0.0128	0.7562
SMOTE	2.5298	0.0439	0.0241	0.8277
ADASYN	2.4670	0.0317	0.0198	0.7639
Mixup	2.4867	0.0459	0.0261	0.8296
GMM	2.6311	0.0472	0.0275	0.8310
HMM_GMM	1.4237	0.0268	0.0070	0.8106
TimeVAE	3.1308	-0.0146	-0.0266	0.8134

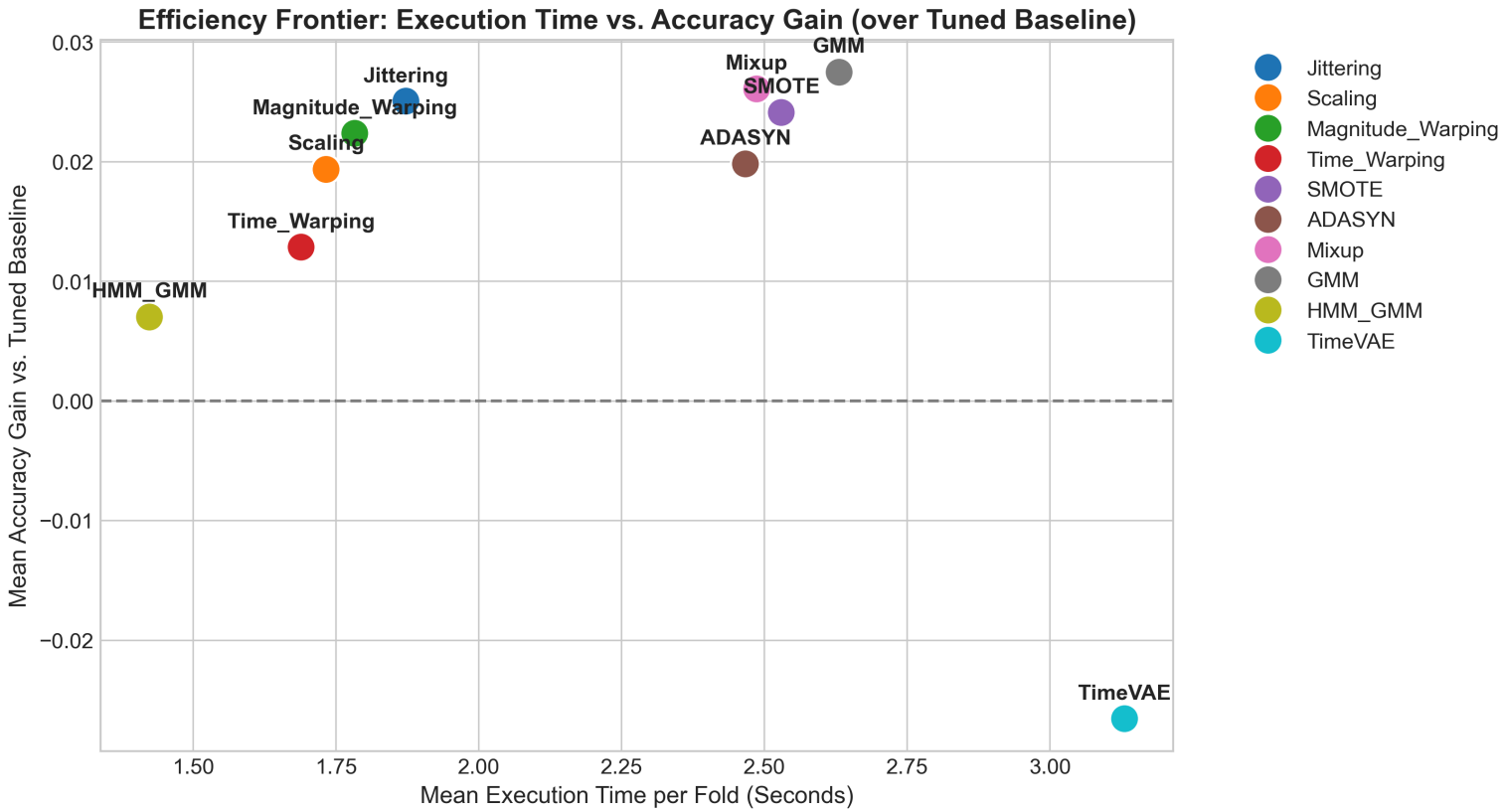


Figure 7.1: Efficiency Frontier — Mean Execution Time (s) vs. Accuracy Gain over Tuned Baseline